

INFO0501

ALGORITHMIQUE AVANCÉE

COURS 4

ARBRES COUVRANTS DE POIDS MINIMAL

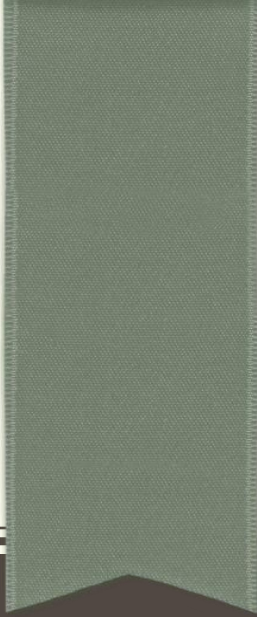


UNIVERSITÉ
DE REIMS
CHAMPAGNE-ARDENNE

Pierre Delisle
Département d'Informatique
Septembre 2022

Plan de la séance

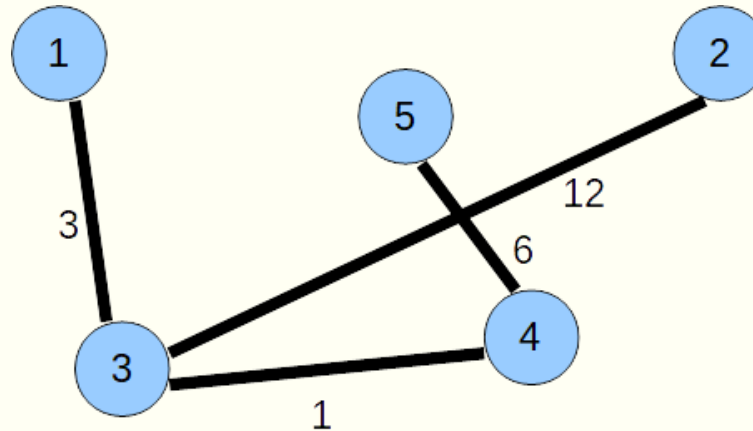
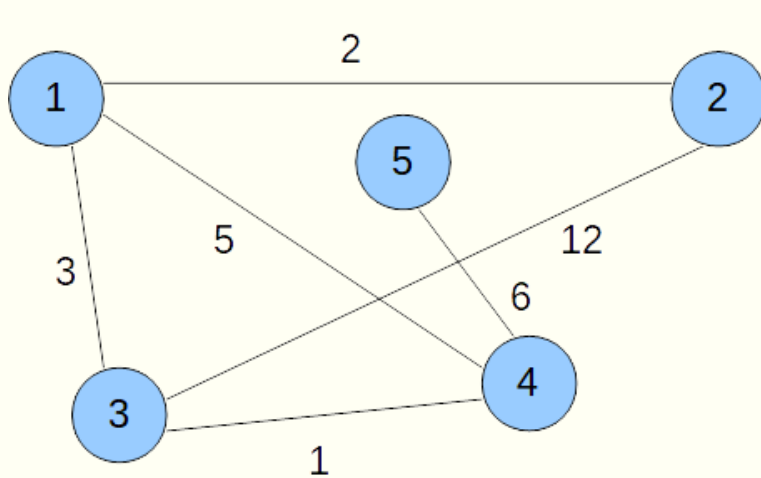
- Arbres couvrants de poids minimal
- Algorithme de Kruskal
 - Structure de données pour ensembles disjoints
- Algorithme de Prim
 - Structure de file de priorités
- Bibliographie
 - T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, "Algorithmique", 3^e édition, Dunod, 2010



PROBLÈME DE L'ARBRE COUVRANT DE POIDS MINIMAL

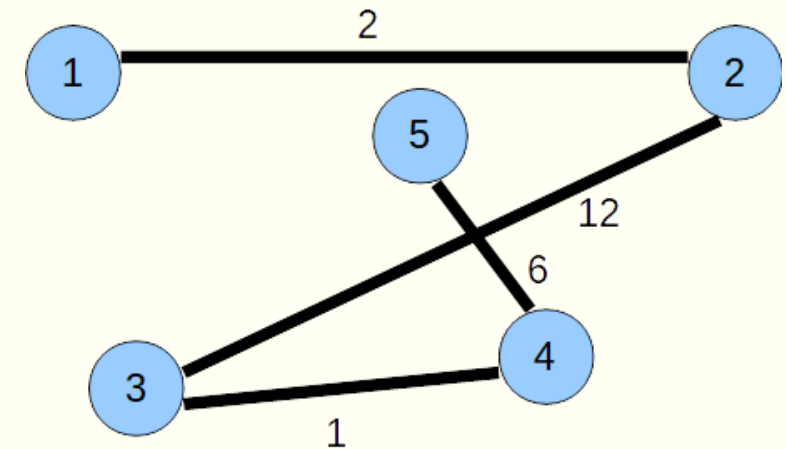
Définition du problème

- Étant donné un graphe non orienté pondéré
- On cherche un graphe partiel de ce graphe
 - ... qui soit un arbre ...
 - ... et qui soit de coût minimum
- Graphe partiel : l'arbre a pour ensemble de sommets tous les sommets du graphe initial
 - On dit que c'est un arbre couvrant
- En anglais : minimum spanning tree
- On suppose le graphe initial connexe



Arbre couvrant
(de poids 22)

Info0501 - Cours 4



Arbre couvrant
(de poids 21)

Applications

- Réseaux
 - On estime le coût des liaisons directes entre toutes les machines à relier
 - Puis on cherche à réaliser un réseau connexe à coût minimum
- Traitement d'images
 - On représente une image sous forme de pixels
 - On veut déterminer des régions dans l'image
 - Graphe : chaque pixel est un sommet à 8 voisins
 - On value les arêtes par la différence de gris
 - On construit l'arbre couvrant minimum, puis on sépare

Construction d'un arbre couvrant minimal

- Soit un graphe $G = (S, A)$
 - Non orienté
 - Connexe
 - Fonction de pondération $p \rightarrow$ attribue un poids $p(u, v)$ à chaque arête (u, v)
- Stratégies gloutonnes
 - On construit l'arbre arête par arête
 - ... en ajoutant à chaque étape une arête appropriée de A

ACM-GÉNÉRIQUE (G, p)

$E = \emptyset$

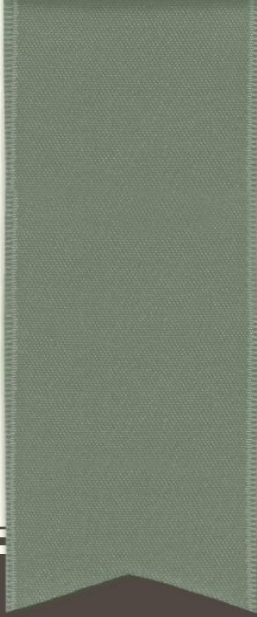
Tant que E ne forme pas un arbre couvrant

Trouver une arête (u, v) appropriée

$E = E \cup (u, v)$

Retourner E

- 2 algorithmes principaux $\rightarrow$ Kruskal et Prim
 - Utilisent cette stratégie gloutonne
 - Utilisent une règle différente pour choisir l'arête (u, v) appropriée



ALGORITHME DE KRUSKAL

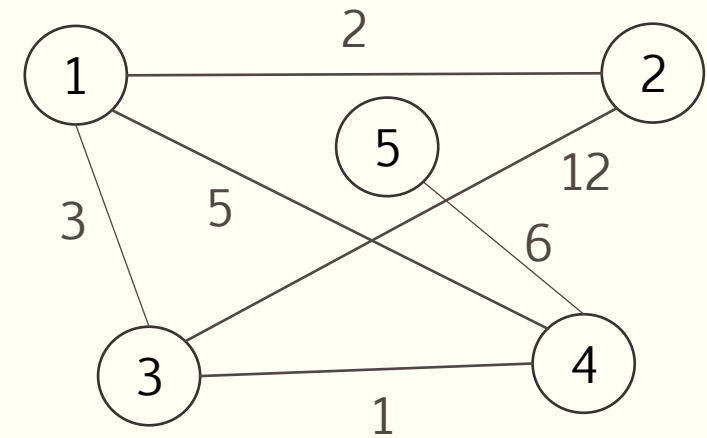
Principe de l'algorithme

- Pour déterminer un arbre couvrant de poids minimal d'un graphe connexe à $|S|$ sommets, ...
- ... on sélectionne les arêtes d'un graphe partiel initialement sans arête ...
- ... en itérant $|S| - 1$ fois l'opération suivante
 - Choisir une arête de poids minimal ne formant pas un cycle avec les arêtes précédemment choisies

Fonctionnement de l'algorithme de Kruskal

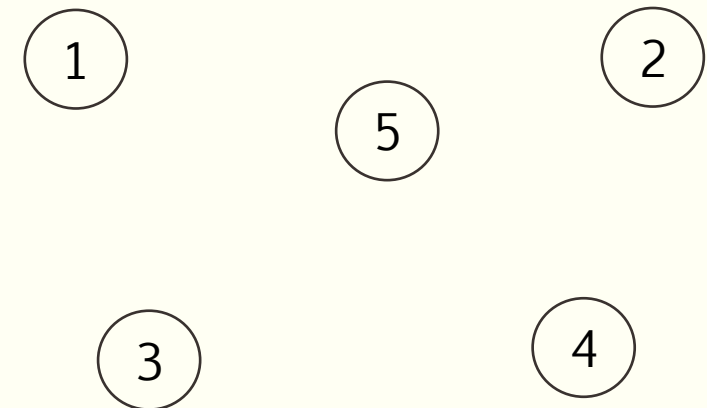
- Graphe initial

- $|S| = 5$



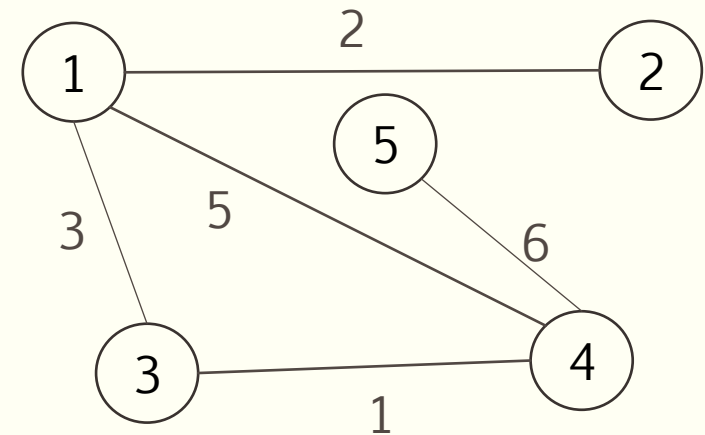
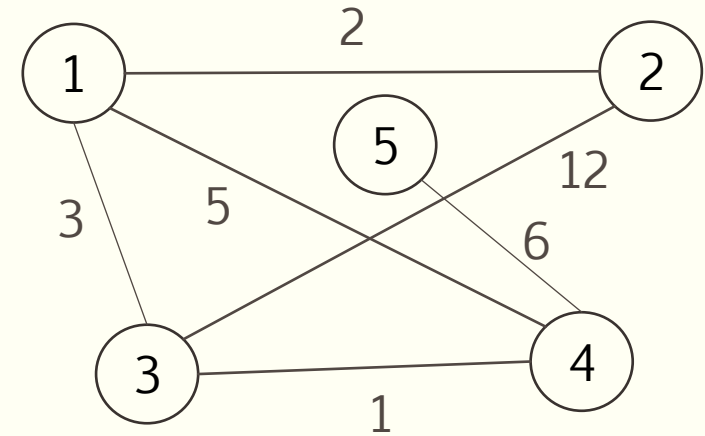
- Graphe partiel initial

- Tous les sommets, aucune arête



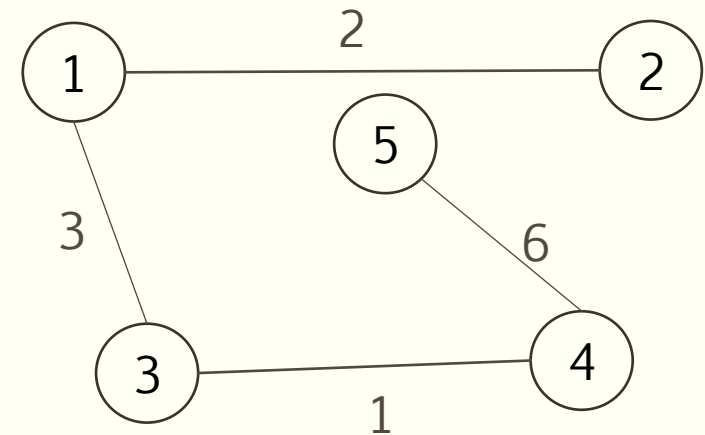
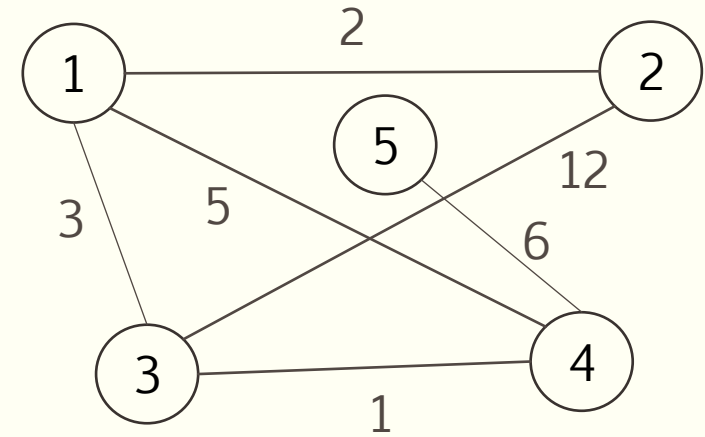
Fonctionnement de l'algorithme de Kruskal

- Graphe initial
 - $|S| = 5$
- On choisit une arête de poids minimal qui ne forme pas de cycle avec les autres arêtes retenues
 - (3, 4)
 - (1, 2)
 - (1, 3)
 - (1, 4) ? Elle forme un cycle, on ne la prend pas
 - (4, 5)



Fonctionnement de l'algorithme de Kruskal

- Graphe initial
 - $|S| = 5$
- On a retenu $|S| - 1$ arêtes : fin de l'algorithme
- On a donc un arbre couvrant de poids minimal de poids 12 ($1 + 2 + 3 + 6$)
 - Qui s'avère être une chaîne dans ce cas-ci (une chaîne est un arbre)



Mise en oeuvre

- Trier les arêtes par ordre de poids croissants
- Tant qu'on n'a pas retenu $|S| - 1$ arêtes
 - Considérer, dans l'ordre du tri, la 1ère arête non examinée
 - Si elle forme un cycle avec celles précédentes, la rejeter, sinon la garder
- Il reste à résoudre un problème
 - Comment déterminer si une arête choisie à l'étape i forme un cycle avec les arêtes précédemment choisies ?

Principe : gérer l'évolution des composantes connexes

- Pour qu'une arête (u, v) ferme un cycle, il faut que ses extrémités aient été précédemment reliées par une chaîne
 - Donc aient été dans une même composante connexe
- Nous allons donc gérer l'évolution des composantes connexes au fur et à mesure du choix des arêtes
 - Par tableau $\rightarrow$ plus simple mais moins efficace
 - Par collection d'ensembles disjoints $\rightarrow$ efficace

Gérer l'évolution des composantes connexes par tableau

- Initialement, le graphe ne contient aucune arête
 - Chaque sommet est une composante connexe
 - On initialise chaque sommet à son indice i dans un tableau de longueur $|S|$
- Chaque fois qu'une arête $\{u, v\}$ est candidate
 - On compare les valeurs aux indices u et v dans le tableau
 - Si valeurs égales, u et v sont dans la même composante connexe $\rightarrow$ ajouter l'arête créerait alors un cycle donc on ne la retient pas
 - Si valeurs différentes, on garde l'arête $\{u, v\}$, on donne à l'indice de v la valeur de l'indice de u , ainsi que tout sommet d'indice v
 - Autrement dit, après sélection de l'arête $\{u, v\}$, tous les sommets de la composante connexe de u et de la composante connexe de v ne forment qu'une seule composante connexe et ont le même indice

Exemple – Mise en œuvre de l'algorithme de Kruskal

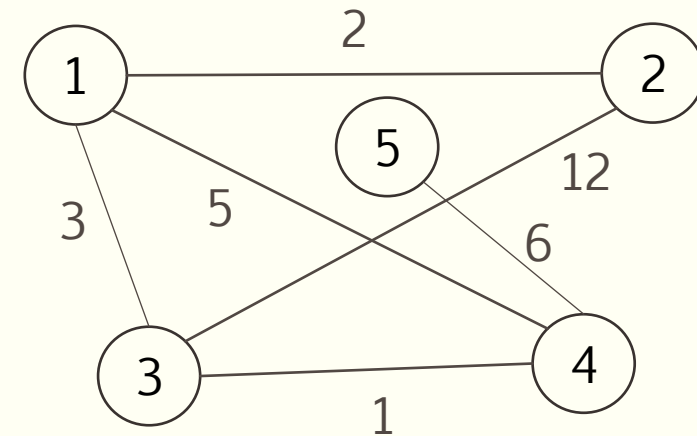
■ Graphe initial

- $|S| = 5$
- $|A| = 6$

■ On crée un tableau d'arêtes

- Et on le trie par ordre de poids croissant

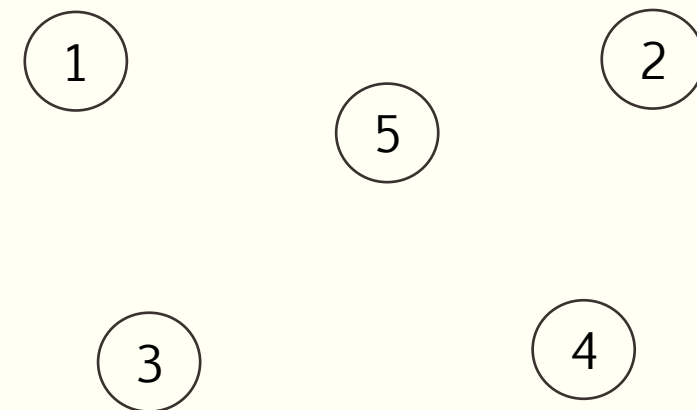
	t		t
1	(1, 2) 2	1	(3, 4) 1
2	(1, 3) 3	2	(1, 2) 2
3	(1, 4) 5	3	(1, 3) 3
4	(2, 3) 12	4	(1, 4) 5
5	(3, 4) 1	5	(4, 5) 6
6	(4, 5) 6	6	(2, 3) 12



■ Graphe partiel initial

- Tous les sommets, aucune arête
- On crée un tableau cc
- On initialise chaque sommet à son indice i : chaque sommet est une composante connexe

	cc		cc
1		1	1
2		2	2
3		3	3
4		4	4
5		5	5



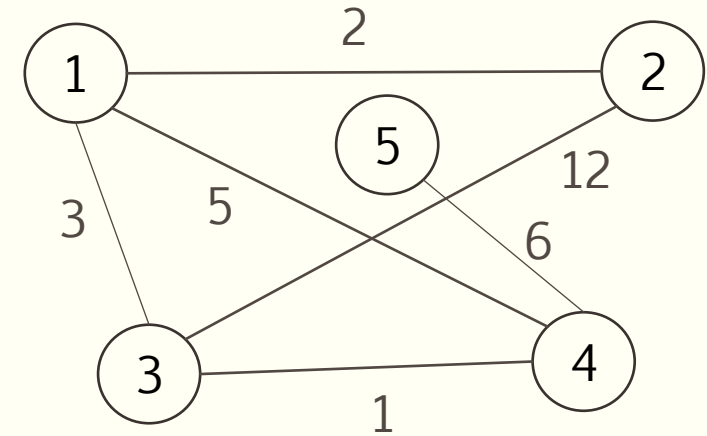
Exemple – Mise en œuvre de l'algorithme de Kruskal

- Graphe initial

- $|S| = 5$
- $|A| = 6$

→

t	
1	(3, 4) 1
2	(1, 2) 2
3	(1, 3) 3
4	(1, 4) 5
5	(4, 5) 6
6	(2, 3) 12



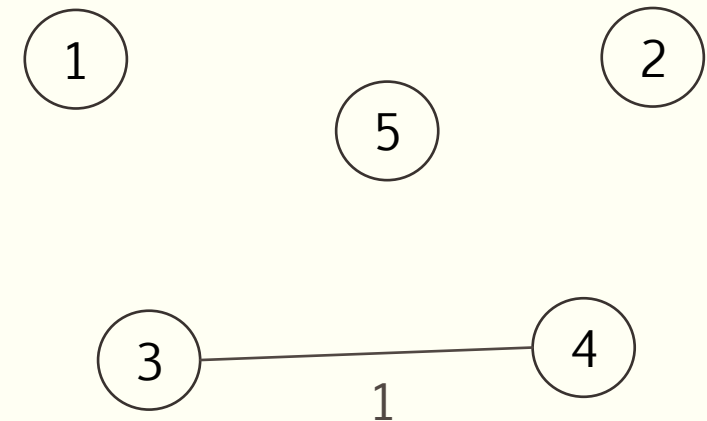
- On prend la 1^{ère} arête dans l'ordre du tri

- $cc[3] \neq cc[4]$
- On retient l'arête
- On met à jour tous les sommets égaux à $cc[4]$ avec $cc[3]$

→

cc	
1	1
2	2
3	3
4	3
5	5

→



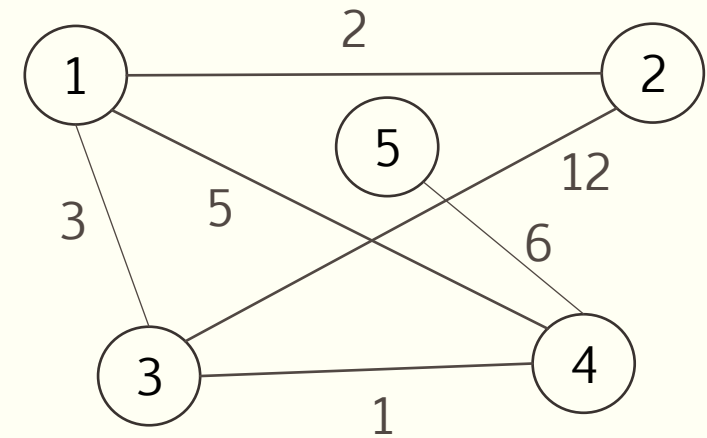
Exemple – Mise en œuvre de l'algorithme de Kruskal

- Graphe initial

- $|S| = 5$
- $|A| = 6$

→

t		
1	(3, 4)	1
2	(1, 2)	2
3	(1, 3)	3
4	(1, 4)	5
5	(4, 5)	6
6	(2, 3)	12

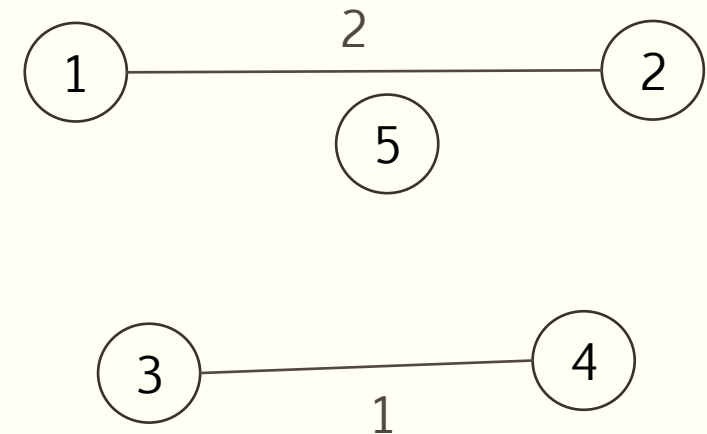


- On prend la 2^e arête dans l'ordre du tri

- $cc[1] \neq cc[2]$
- On retient l'arête
- On met à jour tous les sommets égaux à $cc[2]$ avec $cc[1]$

→

cc	
1	1
2	1
3	3
4	3
5	5



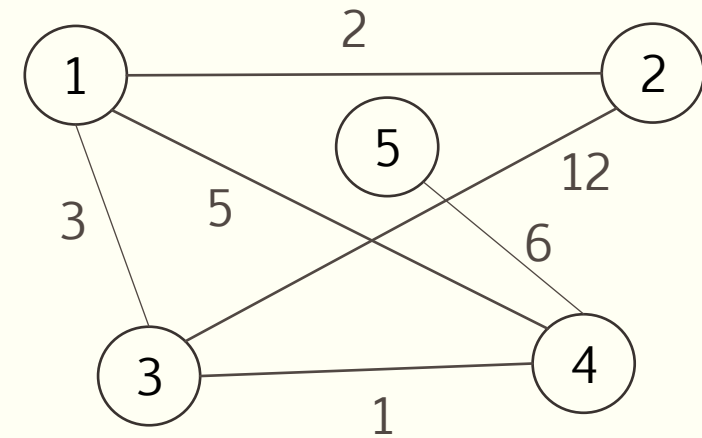
Exemple – Mise en œuvre de l'algorithme de Kruskal

- Graphe initial

- $|S| = 5$
- $|A| = 6$

→

t	
1	(3, 4) 1
2	(1, 2) 2
3	(1, 3) 3
4	(1, 4) 5
5	(4, 5) 6
6	(2, 3) 12



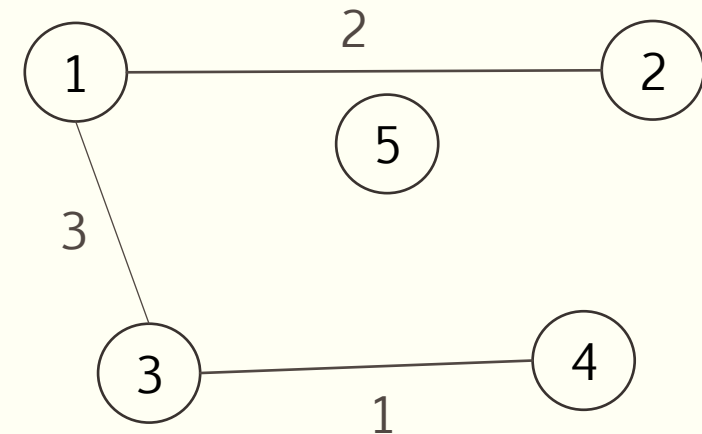
- On prend la 3^e arête dans l'ordre du tri

- $cc[1] \neq cc[3]$
- On retient l'arête
- On met à jour tous les sommets égaux à $cc[3]$ avec $cc[1]$

→

cc	
1	1
2	1
3	3
4	3
5	5

→



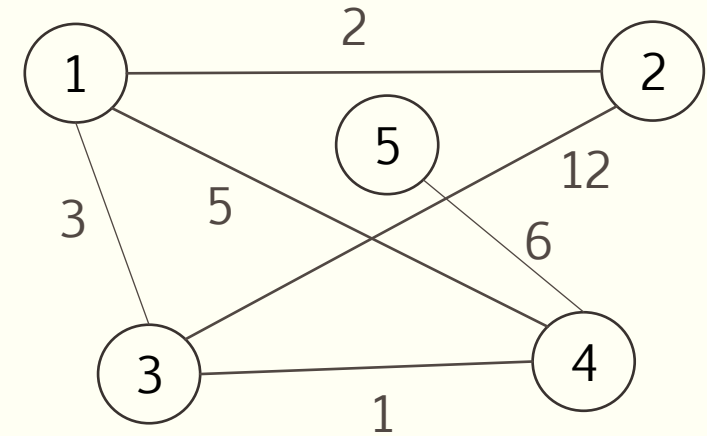
Exemple – Mise en œuvre de l'algorithme de Kruskal

- Graphe initial

- $|S| = 5$
- $|A| = 6$

→

t		
1	(3, 4)	1
2	(1, 2)	2
3	(1, 3)	3
4	(1, 4)	5
5	(4, 5)	6
6	(2, 3)	12



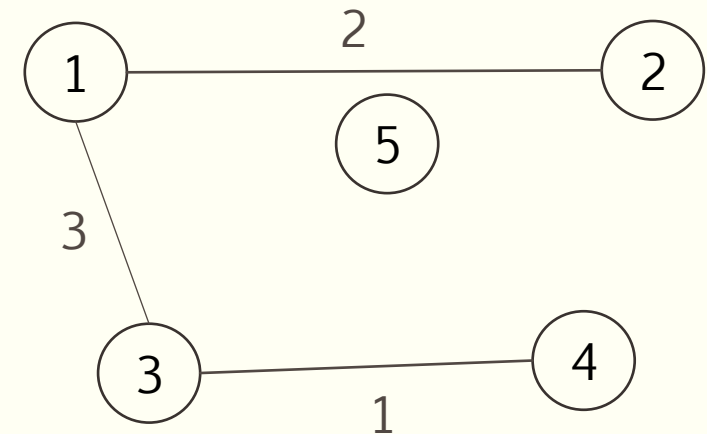
- On prend la 4^e arête dans l'ordre du tri

- $cc[1] == cc[4]$
- On ne retient pas l'arête

→

cc	
1	1
2	1
3	1
4	1
5	5

→



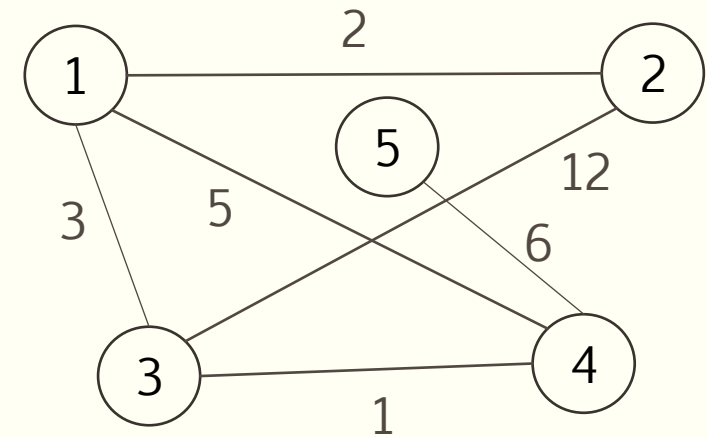
Exemple – Mise en œuvre de l'algorithme de Kruskal

- Graphe initial

- $|S| = 5$
- $|A| = 6$

→

t		
1	(3, 4)	1
2	(1, 2)	2
3	(1, 3)	3
4	(1, 4)	5
5	(4, 5)	6
6	(2, 3)	12



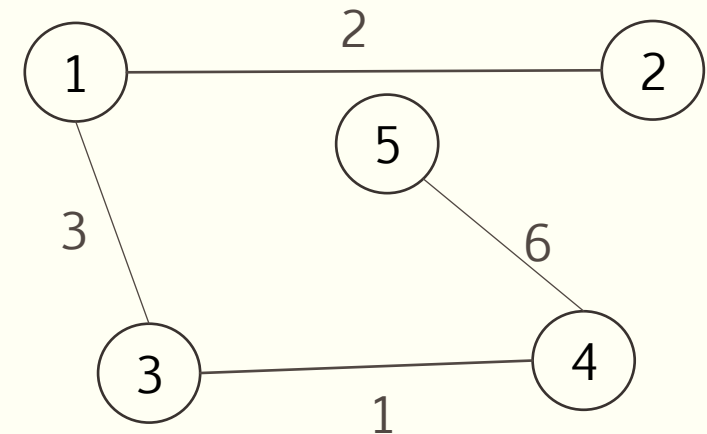
- On prend la 5^e arête dans l'ordre du tri

- $cc[4] \neq cc[5]$
- On retient l'arête
- On met à jour tous les sommets égaux à $cc[5]$ avec $cc[4]$

→

cc	
1	1
2	1
3	1
4	1
5	5

→



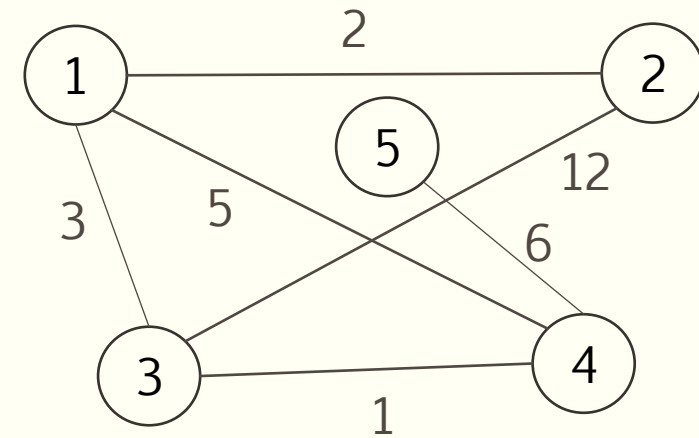
Exemple – Mise en œuvre de l'algorithme de Kruskal

- Graphe initial

- $|S| = 5$
- $|A| = 6$

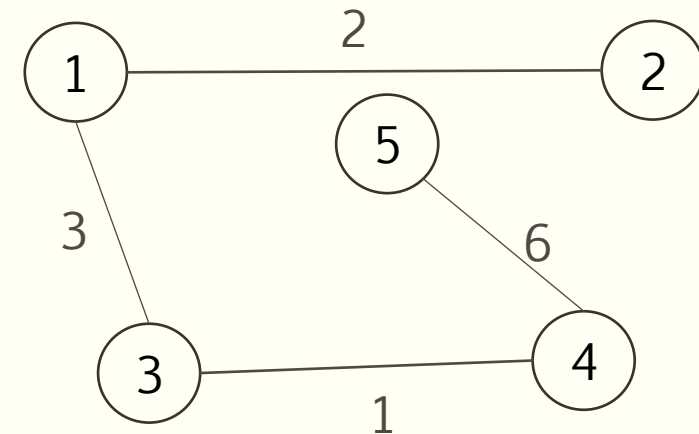
→

t	
1	(3, 4) 1
2	(1, 2) 2
3	(1, 3) 3
4	(1, 4) 5
5	(4, 5) 6
6	(2, 3) 12



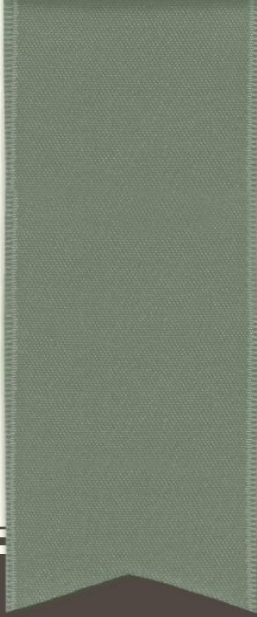
- On a retenu $|S| - 1$ arêtes : fin de l'algorithme
- Tous les sommets sont dans la même composante connexe

CC	
1	1
2	1
3	1
4	1
5	1



Remarques

- Après la sélection d'une arête valide
 - On doit parcourir tout le tableau pour mettre à jour les indices de composantes connexes
 - $\alpha(S)$
- On peut réduire ce temps par l'utilisation d'une structure de données pour ensembles disjoints



STRUCTURES DE DONNÉES POUR ENSEMBLES DISJOINTS

Ensembles disjoints

- On veut regrouper n éléments distincts dans une collection $S = \{S_1, S_2, \dots, S_k\}$ d'ensembles dynamiques disjoints
 - Chaque élément fait partie d'un seul ensemble
 - Chaque ensemble est identifié par un représentant
- Opérations sur les ensembles disjoints
 - CRÉER-ENSEMBLE (x)
 - TROUVER-ENSEMBLE (x)
 - UNION (x, y)

Opérations sur les ensembles disjoints

▪ CRÉER-ENSEMBLE (x)

- À partir d'un pointeur sur un objet x
- Crée un nouvel ensemble dont le seul membre (et le représentant) est x
- Ensembles disjoints
 - x ne doit pas être déjà membre d'un autre ensemble

▪ TROUVER-ENSEMBLE (x)

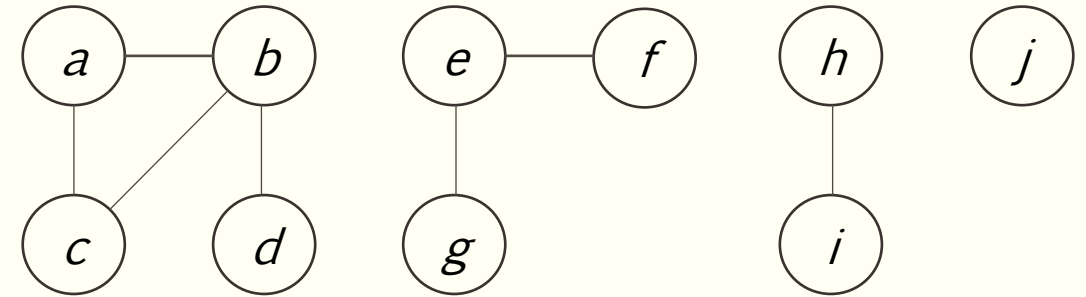
- Retourne un pointeur vers le représentant de l'ensemble contenant x

▪ UNION (x, y)

- Réunit les ensembles dynamiques qui contiennent x et y ...
- ... appelés S_x et S_y
- ... dans un nouvel ensemble qui est l'union de S_x et S_y
- Le représentant du nouvel ensemble sera un élément quelconque d'un des 2 ensembles
- Détruit les ensembles initiaux S_x et S_y

Exemple d'application d'une structure d'ensembles disjoints

- On veut déterminer les composantes connexes d'un graphe non orienté
- On va traiter chaque arête du graphe
 - Si l'arête (i, j) existe, alors i et j sont nécessairement dans la même composante connexe

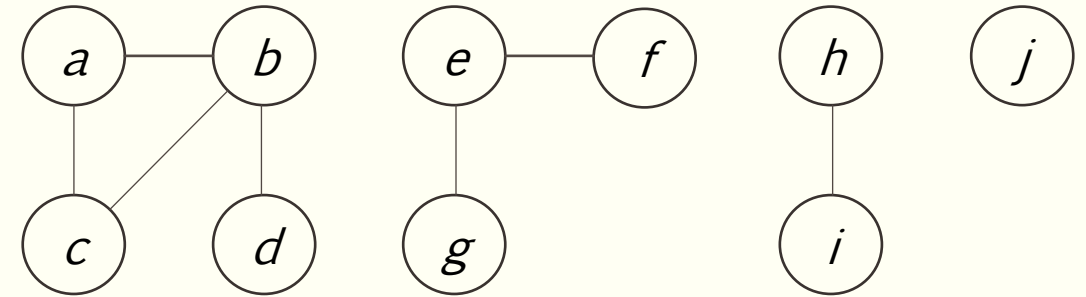


- Ensembles disjoints initiaux
 - Création d'un ensemble par sommet
- On traite ensuite chaque arête
 - Si le sommet d'origine et le sommet d'extrémité ne font pas partie du même ensemble, on fusionne les 2 ensembles

Arêtes traitées	Collection d'ensembles disjoints
Ensembles initiaux	{a} {b} {c} {d} {e} {f} {g} {h} {i} {j}
(b, d)	{a} {b, d} {c} {e} {f} {g} {h} {i} {j}
(e, g)	{a} {b, d} {c} {e, g} {f} {h} {i} {j}
(a, c)	{a, c} {b, d} {e, g} {f} {h} {i} {j}
(h, i)	{a, c} {b, d} {e, g} {f} {h, i} {j}
(a, b)	{a, b, c, d} {e, g} {f} {h, i} {j}
(e, f)	{a, b, c, d} {e, f, g} {h, i} {j}
(b, c)	{a, b, c, d} {e, f, g} {h, i} {j}

Exemple d'application d'une structure d'ensembles disjoints

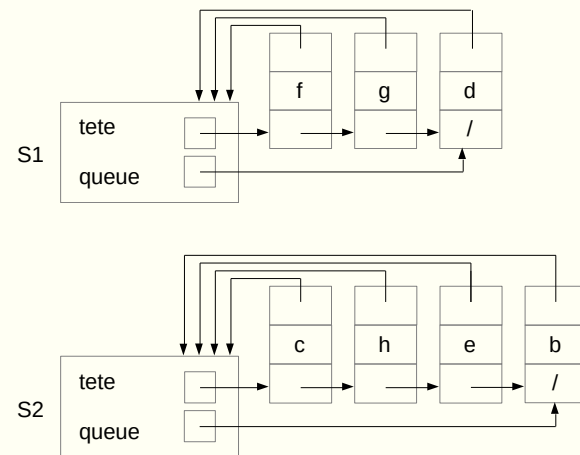
- On veut déterminer les composantes connexes d'un graphe non orienté
- On va traiter chaque arête du graphe
 - Si l'arête (i, j) existe, alors i et j sont nécessairement dans la même composante connexe
- Quand toutes les arêtes ont été traitées, 2 sommets se trouvent dans la même composante connexe si les objets correspondants se trouvent dans le même ensemble
- Fonctions à définir
 - $\text{composantes-connexes}(G)$
 - $\text{même-composante}(u, v)$



Arêtes traitées	Collection d'ensembles disjoints
Ensembles initiaux	{a} {b} {c} {d} {e} {f} {g} {h} {i} {j}
(b, d)	{a} {b, d} {c} {e} {f} {g} {h} {i} {j}
(e, g)	{a} {b, d} {c} {e, g} {f} {h} {i} {j}
(a, c)	{a, c} {b, d} {e, g} {f} {h} {i} {j}
(h, i)	{a, c} {b, d} {e, g} {f} {h, i} {j}
(a, b)	{a, b, c, d} {e, g} {f} {h, i} {j}
(e, f)	{a, b, c, d} {e, f, g} {h, i} {j}
(b, c)	{a, b, c, d} {e, f, g} {h, i} {j}

Représentation d'ensembles disjoints

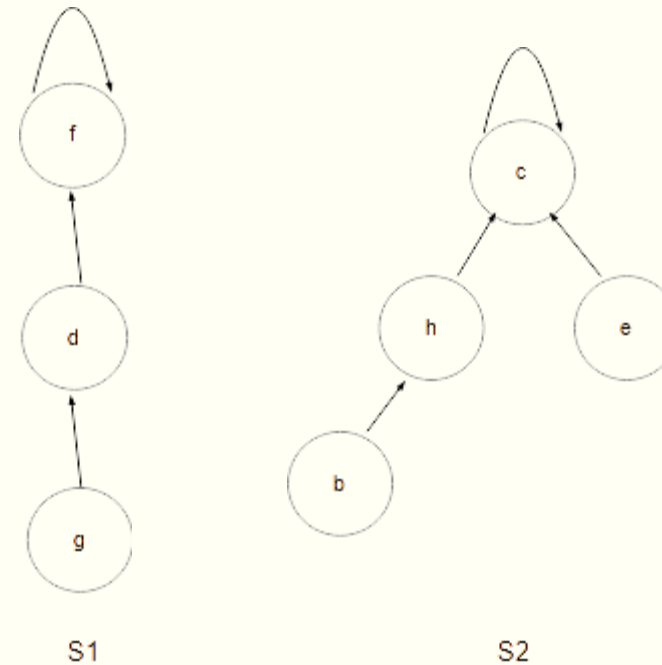
- Par listes chaînées



- Fonctions à définir et temps d'exécution

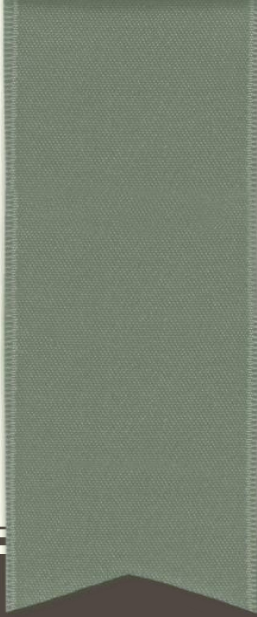
- Créer-ensemble ?
- Trouver-ensemble ?
- Union ?

- Par arbres



Algorithme de Kruskal avec utilisation d'ensembles disjoints

- On crée un ensemble disjoint pour chaque sommet de G
- On trie les arêtes de G
- Pour chaque arête prise par ordre croissant
 - Si l'origine et l'extrémité ne font pas partie du même ensemble, conserver l'arête et réunir les deux ensembles
- Les arêtes retenues constituent alors un arbre couvrant de poids minimal de G



ALGORITHME DE PRIM

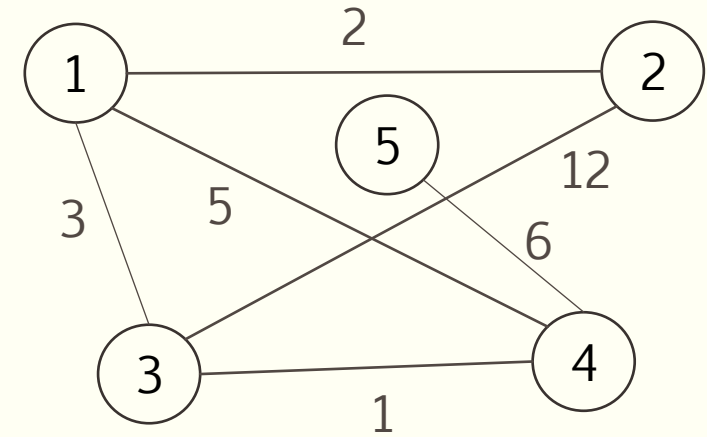
Principe de l'algorithme

- Permet de trouver un arbre couvrant de poids minimum sans devoir trier les arêtes
- On étend, de proche en proche, un arbre couvrant des parties des sommets du graphe
 - En atteignant un sommet supplémentaire à chaque étape ...
 - ... en prenant l'arête la plus légère parmi celles qui joignent l'ensemble des sommets déjà couverts ...
 - ... à l'ensemble des sommets non encore couverts
- Un tableau de distances d permet de connaître la distance entre chaque sommet non couvert et l'ensemble des sommets couverts

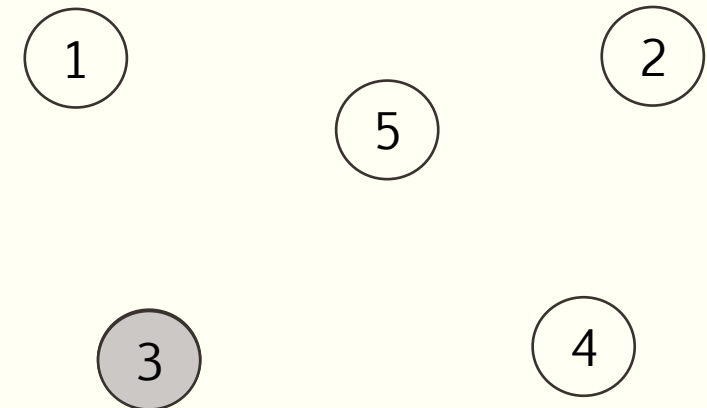
Fonctionnement de l'algorithme de Prim

- Graphe initial

- $|S| = 5$



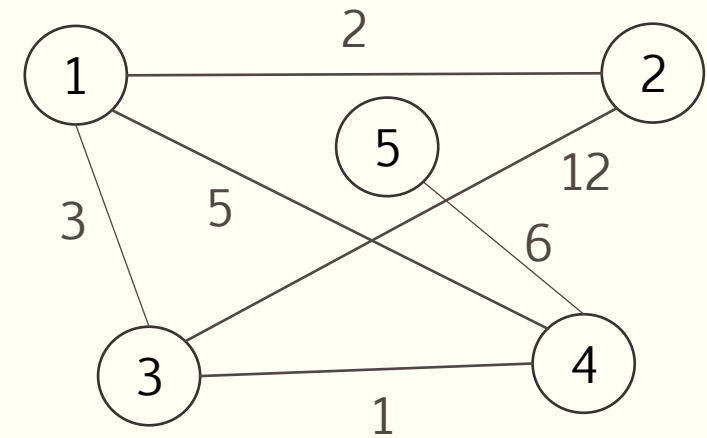
- Au départ, aucun des sommets n'est dans l'arbre en cours de construction
- On démarre d'un sommet quelconque
 - Ici 3
 - On l'ajoute à l'arbre en cours de construction



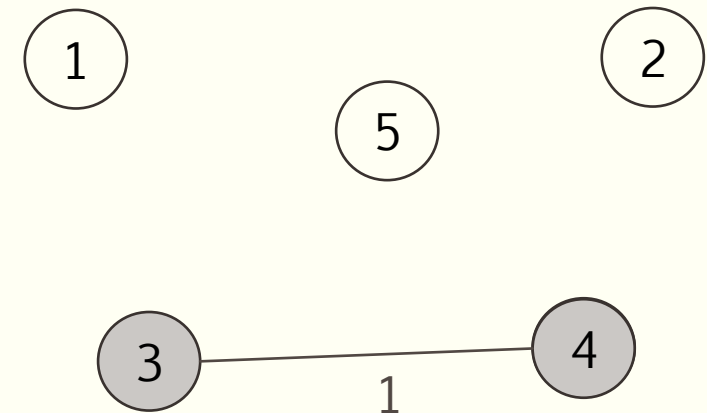
Fonctionnement de l'algorithme de Prim

- Graphe initial

- $|S| = 5$



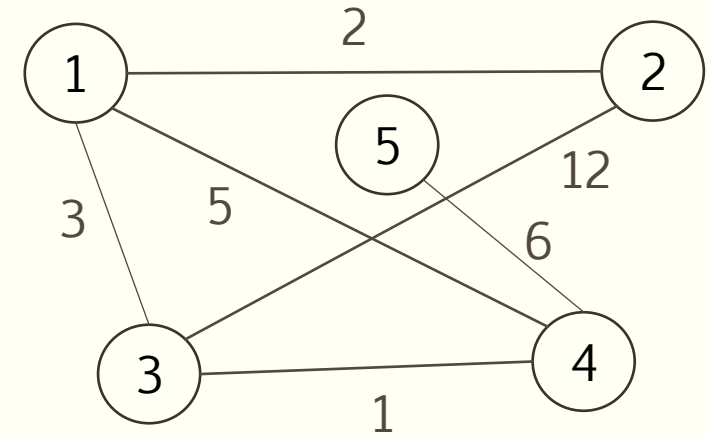
- On choisit ensuite le sommet extérieur le plus près de l'arbre en cours de construction
 - Ici 4
 - On l'ajoute le sommet (4) et l'arête utilisée (3, 4) à l'arbre en cours de construction



Fonctionnement de l'algorithme de Prim

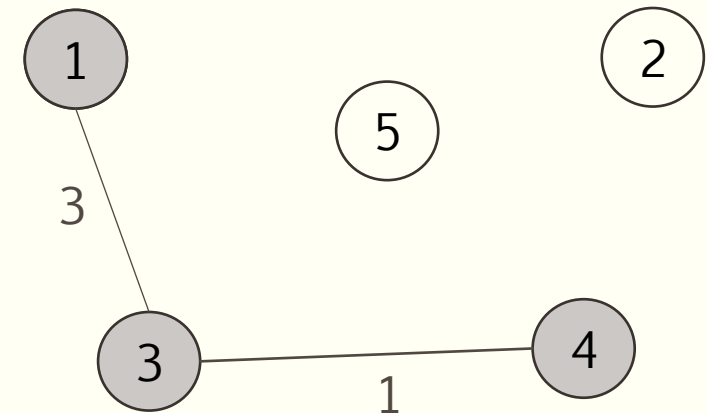
- Graphe initial

- $|S| = 5$



- On choisit ensuite le sommet extérieur le plus près de l'arbre en cours de construction

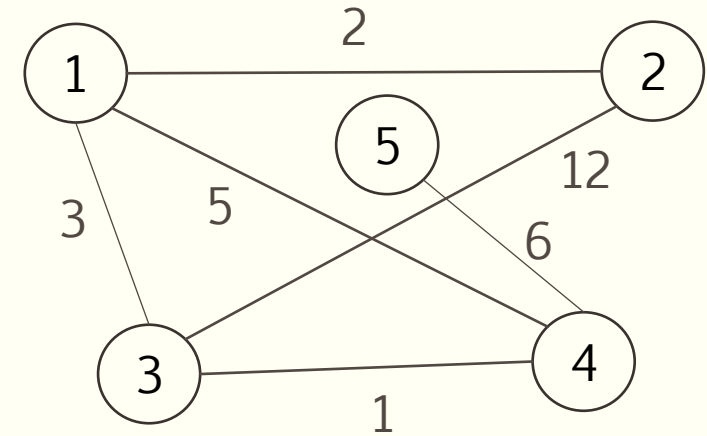
- Ici 1
 - On l'ajoute le sommet (1) et l'arête utilisée (1, 3) à l'arbre en cours de construction



Fonctionnement de l'algorithme de Prim

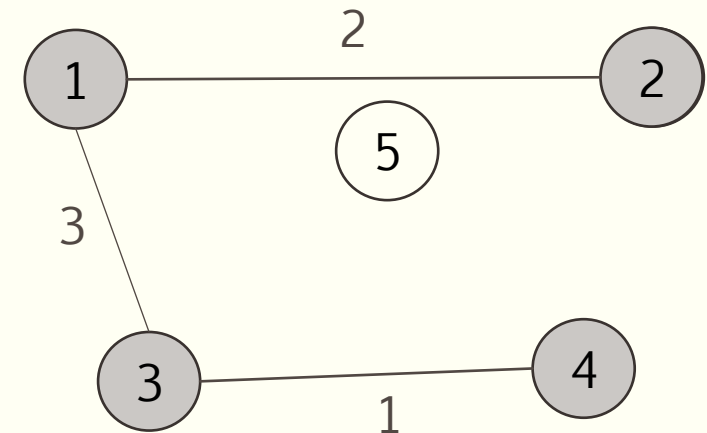
- Graphe initial

- $|S| = 5$



- On choisit ensuite le sommet extérieur le plus près de l'arbre en cours de construction

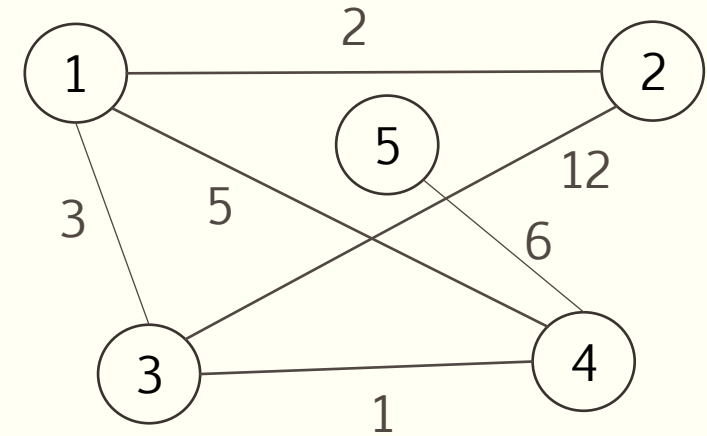
- Ici 2
 - On l'ajoute le sommet (2) et l'arête utilisée (1, 2) à l'arbre en cours de construction



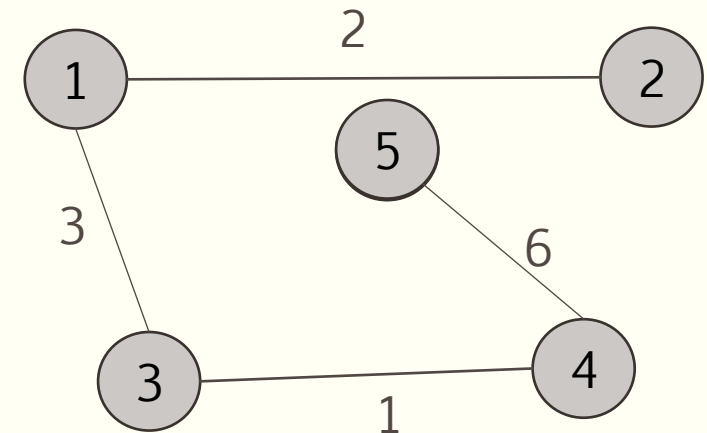
Fonctionnement de l'algorithme de Prim

- Graphe initial

- $|S| = 5$

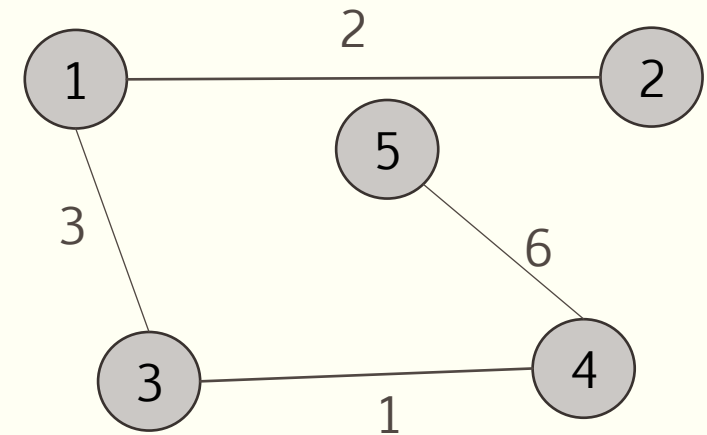
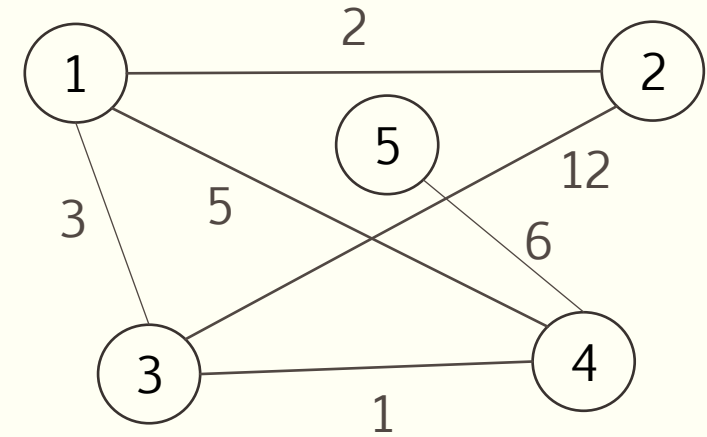


- On choisit ensuite le sommet extérieur le plus près de l'arbre en cours de construction
 - Ici 5
 - On l'ajoute le sommet (5) et l'arête utilisée (4, 5) à l'arbre en cours de construction



Fonctionnement de l'algorithme de Prim

- Graphe initial
 - $|S| = 5$
- Tous les sommets ont été ajoutés à l'arbre en cours de construction : fin de l'algorithme
- On a donc un arbre couvrant de poids minimal de poids 12 ($1 + 3 + 2 + 6$)
 - (Le même que Kruskal, mais les deux algorithmes ne construisent pas toujours le même arbre couvrant de poids minimal)



Mise en œuvre en utilisant des tableaux

- Tableau *d*
 - Distance de chaque sommet extérieur de l'arbre en cours de construction
 - Durant l'exécution de l'algorithme : distance provisoire qui peut être améliorée jusqu'à ce que le sommet soit sélectionné
- Tableau *pere*
 - Permet de reconstituer l'arbre à la fin
 - Indique le père de chaque sommet dans l'arbre couvrant
 - Durant l'exécution de l'algorithme : père provisoire, modifié lorsque la distance provisoire est améliorée
- Tableau *couvert*
 - Permet de savoir si un sommet fait partie de l'arbre en cours de construction ou non

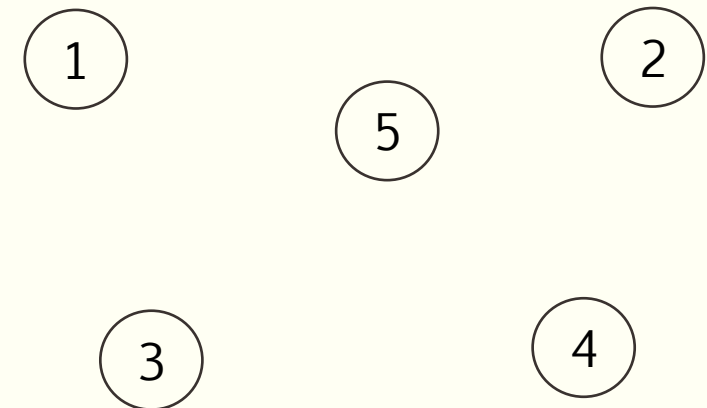
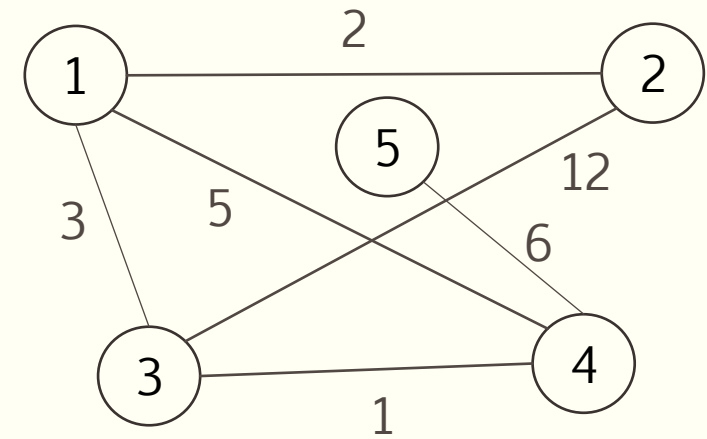
Exemple – Mise en œuvre de l'algorithme de Prim

- Graphe initial
 - $|S| = 5$
- On crée et initialise les tableaux
 - d
 - $pere$
 - $couvert$
- On démarre d'un sommet quelconque
 - Ici 3
 - $d[3] = 0$

d	1	2	3	4	5
	∞	∞	0	∞	∞

$pere$	1	2	3	4	5
	-1	-1	-1	-1	-1

$couvert$	1	2	3	4	5
	F	F	F	F	F

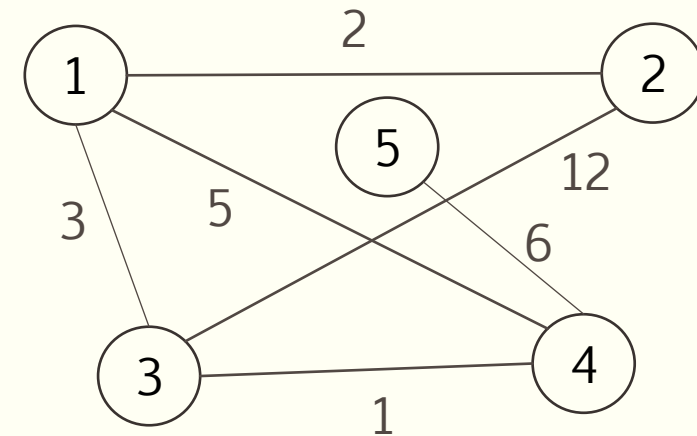


Exemple – Mise en œuvre de l'algorithme de Prim

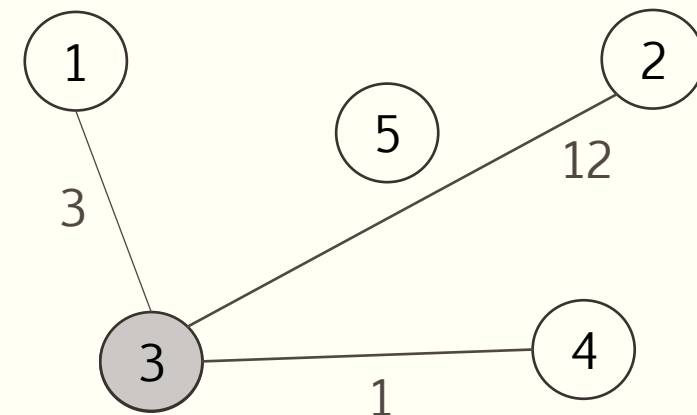
- Graphe initial

- $|S| = 5$

	1	2	3	4	5
<i>d</i>	3	12	0	1	∞
	1	2	3	4	5
<i>pere</i>	-1	-1	-1	-1	-1
	1	2	3	4	5
<i>couvert</i>	F	F	V	F	F



- On choisit ensuite le sommet extérieur le plus près de l'arbre en cours de construction
- Ce qui revient à choisir dans *d* le sommet de distance minimale non encore couvert
 - Ici 3 : on l'ajoute à l'arbre en cours de construction
 - *couvert* [3] = Vrai
 - Puis on parcourt ses voisins 1, 2 et 4
 - Si la distance entre lui et son voisin est inférieure à la distance provisoire du voisin, on met à jour la distance provisoire et le père du voisin

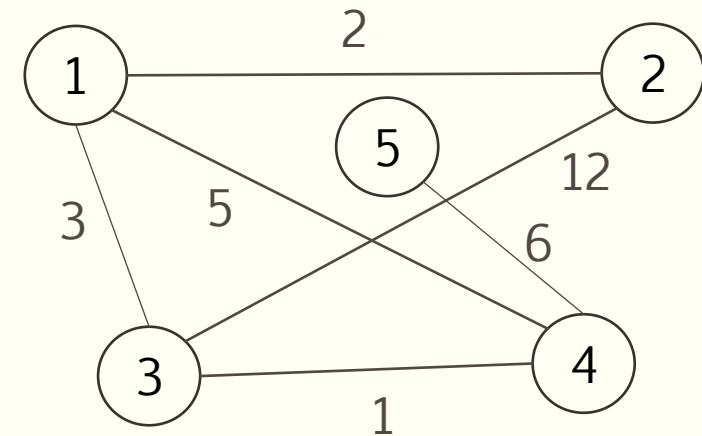


Exemple – Mise en œuvre de l'algorithme de Prim

- Graphe initial

- $|S| = 5$

<i>d</i>	1	2	3	4	5
	3	12	0	1	6
<i>pere</i>	1	2	3	4	5
	3	3	-1	3	4
<i>couvert</i>	1	2	3	4	5
	V	F	V	V	F

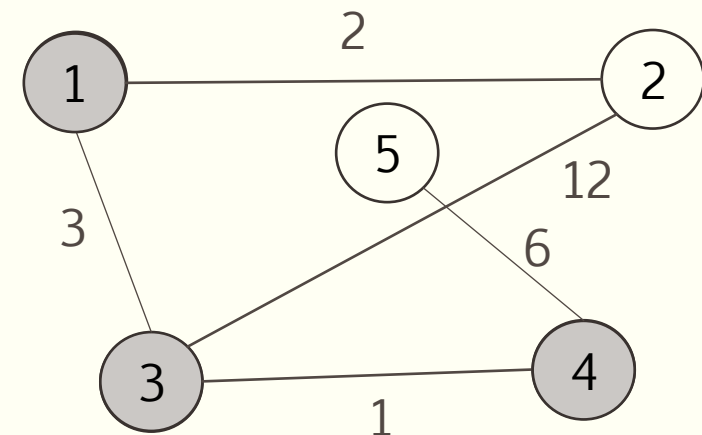


- On choisit ensuite le sommet non couvert de distance minimale

- Ici 4 : *couvert* [4] = Vrai
 - Puis on met éventuellement à jour ses voisins

- On choisit ensuite le sommet non couvert de distance minimale

- Ici 1 : *couvert* [1] = Vrai
 - Puis on met éventuellement à jour ses voisins (on a amélioré la distance provisoire de 2)

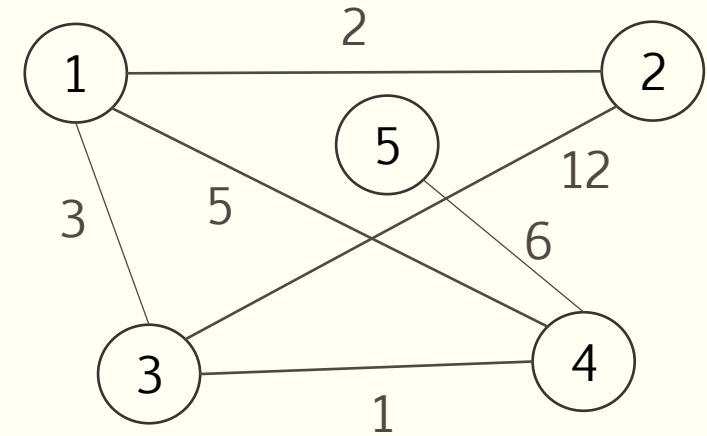


Exemple – Mise en œuvre de l'algorithme de Prim

- Graphe initial

- $|S| = 5$

<i>d</i>	1	2	3	4	5
	3	2	0	1	6
<i>pere</i>	1	2	3	4	5
	3	1	-1	3	4
<i>couvert</i>	1	2	3	4	5
	V	V	V	V	V

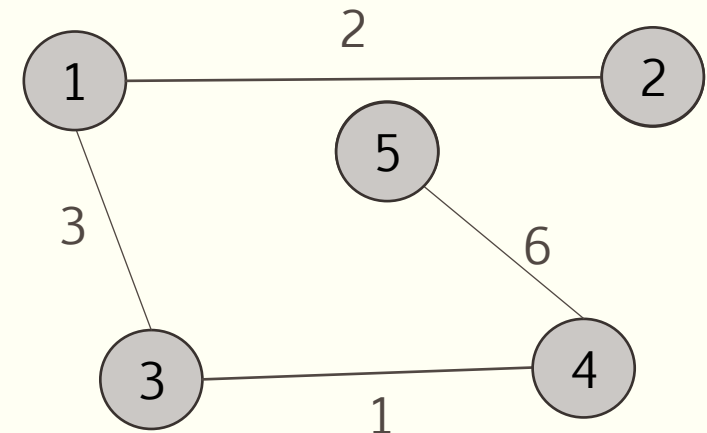


- On choisit ensuite le sommet non couvert de distance minimale

- Ici 2 : *couvert* [2] = Vrai
- Aucun changement sur les voisins

- On choisit ensuite le sommet non couvert de distance minimale

- Ici 5 : *couvert* [5] = Vrai
- Aucun changement sur les voisins

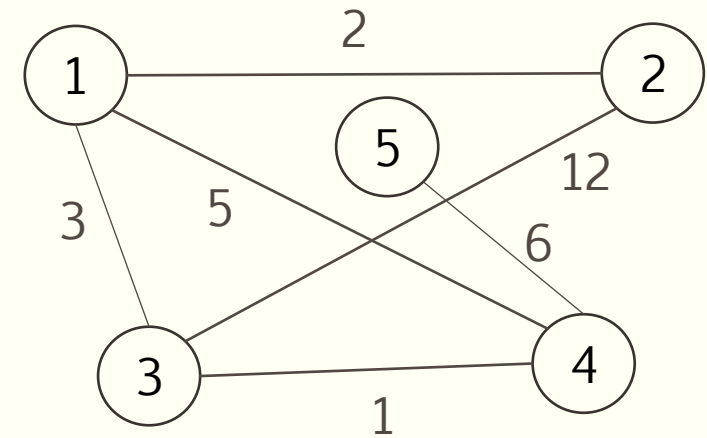


Exemple – Mise en œuvre de l'algorithme de Prim

- Graphe initial

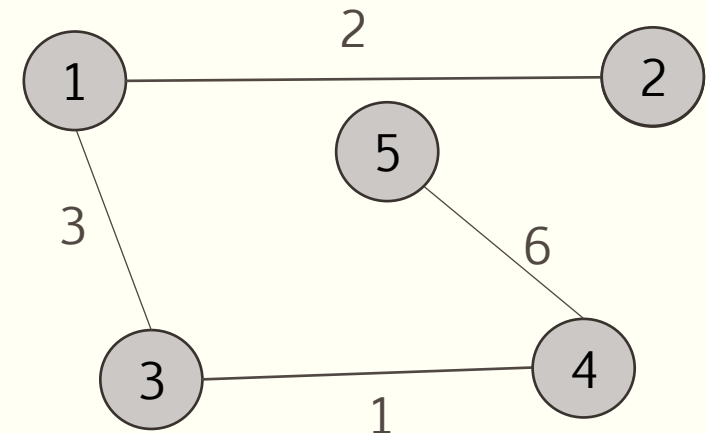
- $|S| = 5$

<i>d</i>	1	2	3	4	5
	3	2	0	1	6
<i>pere</i>	1	2	3	4	5
	3	1	-1	3	4
<i>couvert</i>	1	2	3	4	5
	V	V	V	V	V



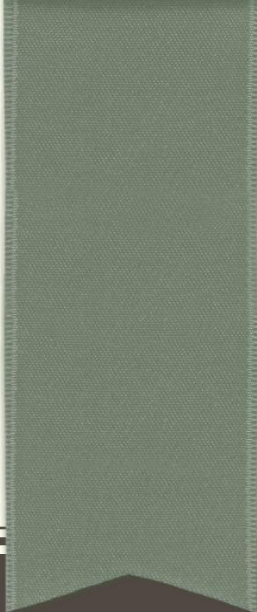
- Tous les sommets ont été ajoutés à l'arbre en cours de construction : fin de l'algorithme

- Arbre couvrant de poids minimal : valeurs de *pere*
- Poids de l'arbre : somme des valeurs de *d*



Remarques

- Après la mise à jour des distances et pères des voisins
 - On doit parcourir tout le tableau d pour déterminer le sommet à ajouter à l'itération suivante
 - $\alpha(S)$
- On peut réduire ce temps par l'utilisation d'une file de priorités min implémentée par tas



FLASHBACK

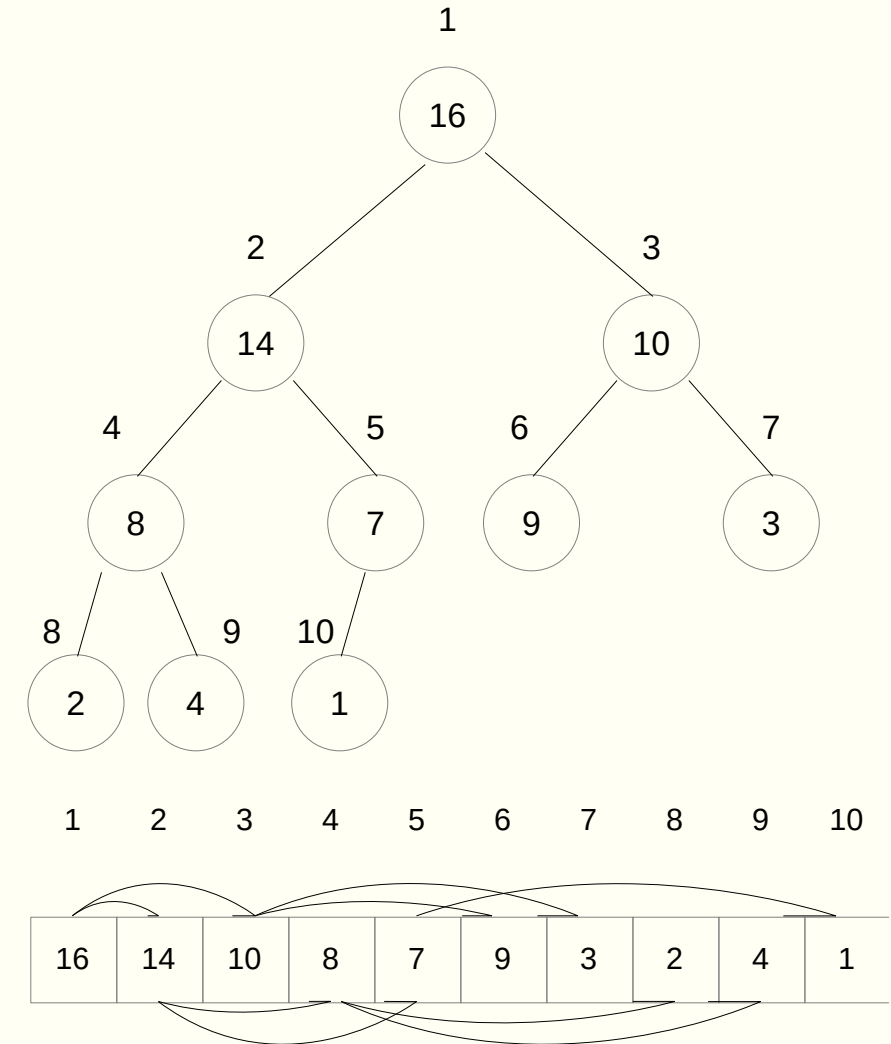
Tas

Tas (binaire)

- Tableau ayant des caractéristiques spécifiques sur l'emplacement des éléments
 - Peut être vu (mais non stocké) comme un arbre binaire presque complet
- Plusieurs utilisations
 - Tri
 - Files de priorités
 - ...

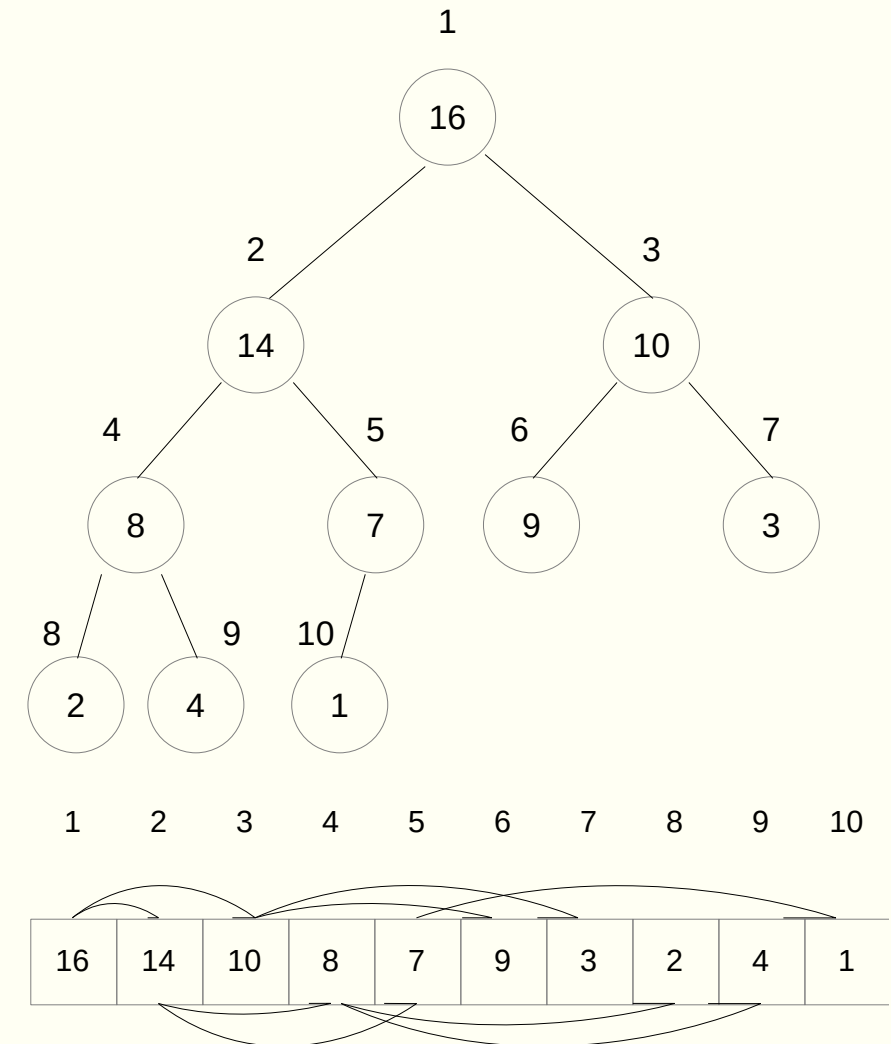
Tas (binaire)

- Chaque nœud de l'arbre correspond à un élément du tableau
- L'arbre est complètement rempli à tous les niveaux
 - Sauf éventuellement le dernier qui est rempli de gauche à droite
- Un tableau t représentant un tas possède 2 attributs
 - **longueur** : nombre maximum d'éléments
 - **taille** : nombre d'éléments du tas effectivement rangés dans le tableau
- Éléments valides du tas $\rightarrow t[1 .. t.taille]$



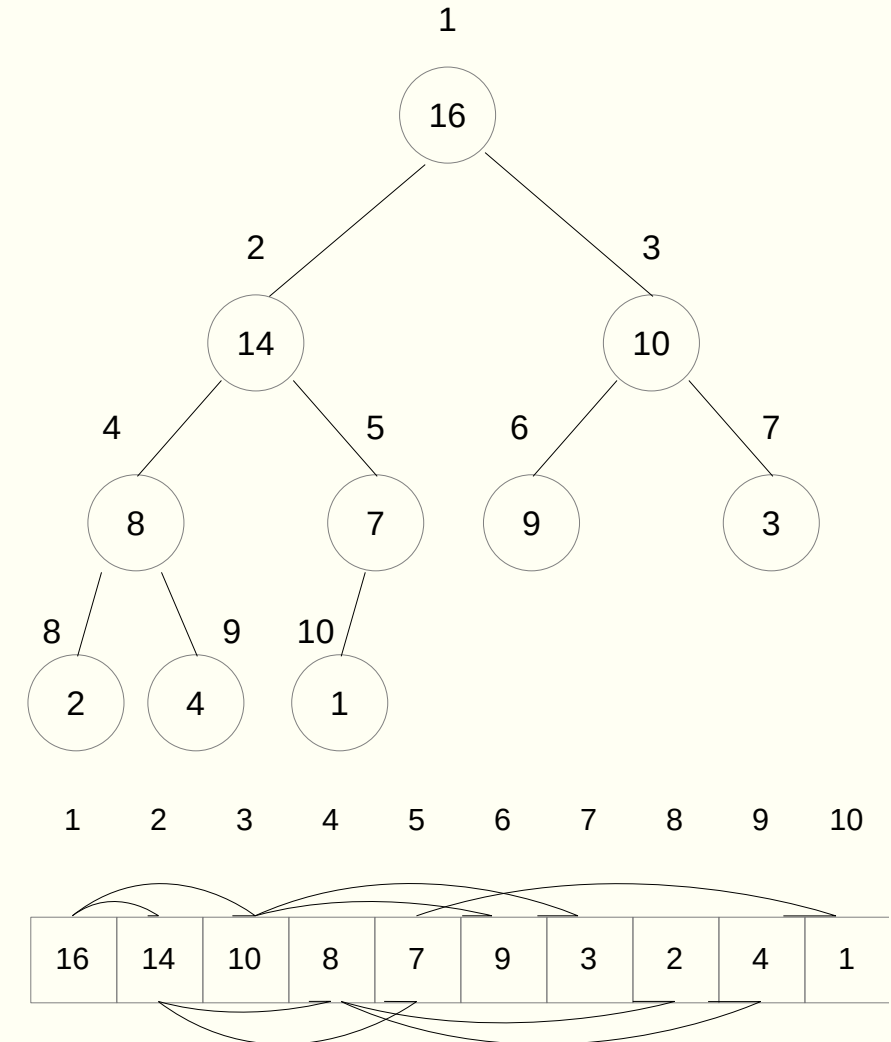
Tas (binaire)

- Racine de l'arbre
 - $t[1]$
- Étant donné l'indice i d'un nœud, on peut calculer
 - L'indice de son parent
 - $\lfloor i / 2 \rfloor$
 - L'indice de son enfant de gauche
 - $2i$
 - L'indice de son enfant de droite
 - $2i + 1$
- Fonctions à définir
 - $\text{parent}(i)$
 - $\text{gauche}(i)$
 - $\text{droite}(i)$



Tas (binaire)

- Propriété de tas (ici pour un tas max)
 - Pour chaque nœud i autre que la racine
 - La valeur d'un nœud est au plus égale à celle du parent
 - $t[\text{parent}(i)] \geq t[i]$
- Plus grand élément stocké à la racine
- Hauteur d'un nœud
 - Nombre d'arcs sur le chemin le plus long reliant le nœud à une feuille
- Hauteur d'un tas de n éléments
 - Hauteur de la racine
 - $\Theta(\lg n)$

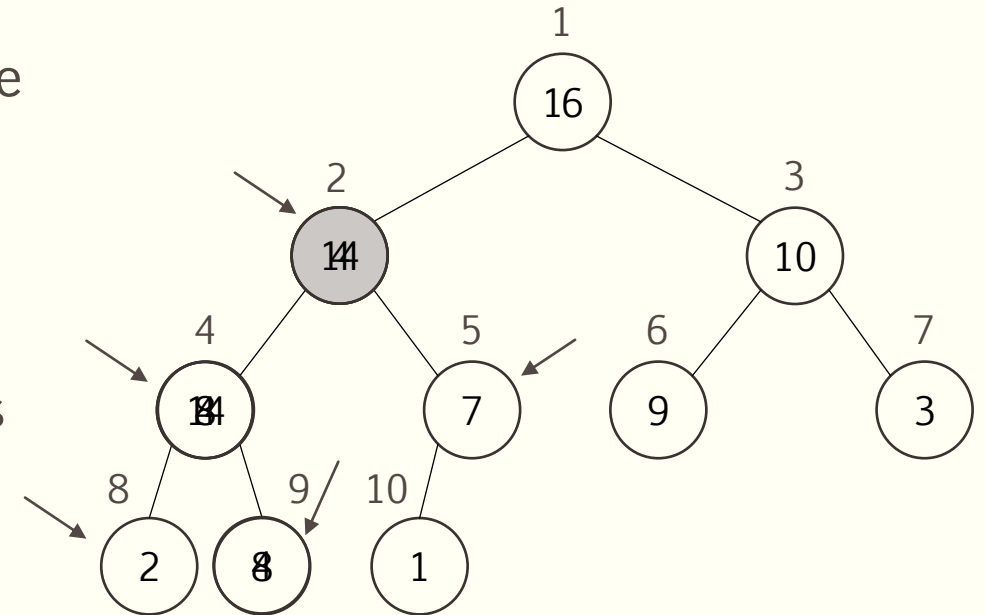


Conservation de la propriété de tas

- Une modification du tableau doit assurer la conservation de la propriété de tas
- Procédure Entasser-Max(t, i)
 - Suppose que les arbres binaires enracinés en GAUCHE(i) et DROITE(i) sont des tas max
 - ... mais que $t[i]$ puisse être plus petit que ses enfants
 - ... violant ainsi la propriété de tas max
 - On fait alors descendre la valeur de $t[i]$ dans le tas max de sorte à rétablir la propriété de tas
- Fonctionnement de Entasser-Max
 - À chaque étape, on détermine le plus grand des éléments aux indices i , GAUCHE(i) et DROITE(i)
 - Si $t[i]$ est le max, on a déjà un tas max
 - Sinon, on échange $t[i]$ avec l'enfant qui est le max et on rappelle Entasser-Max récursivement

Exemple : Entasser-Max avec $t = \langle 16, 4, 10, 14, 7, 9, 3, 2, 8, 1 \rangle$

- Le nœud $t[2]$ de valeur 4 viole la propriété de tas max
- $t[4]$ et $t[5]$ sont des tas max
 - On peut donc entasser
- On trouve le maximum entre le nœud, son fils gauche et son fils droit $\rightarrow$ indice max
- On échange $t[2]$ avec $t[max]$, ici $t[4]$
- Par contre, la propriété de tas max est violée pour $t[4]$
 - On appelle donc récursivement entasser-max à partir de ce nœud
 - Ce qui échange $t[4]$ avec $t[9]$, « répare » le nœud et « répare » l'arbre complètement



- La récursivité se termine lorsqu'il n'y a pas d'échange réalisé
 - Ce qui devient automatique lorsque $GAUCHE(i)$ et $DROITE(i)$ retournent des indices $>$ $t.longueur$
- Fonction à définir : Entasser-Max
- Temps d'exécution proportionnel à la hauteur de l'arbre
 - $O(\lg n)$

Construction d'un tas

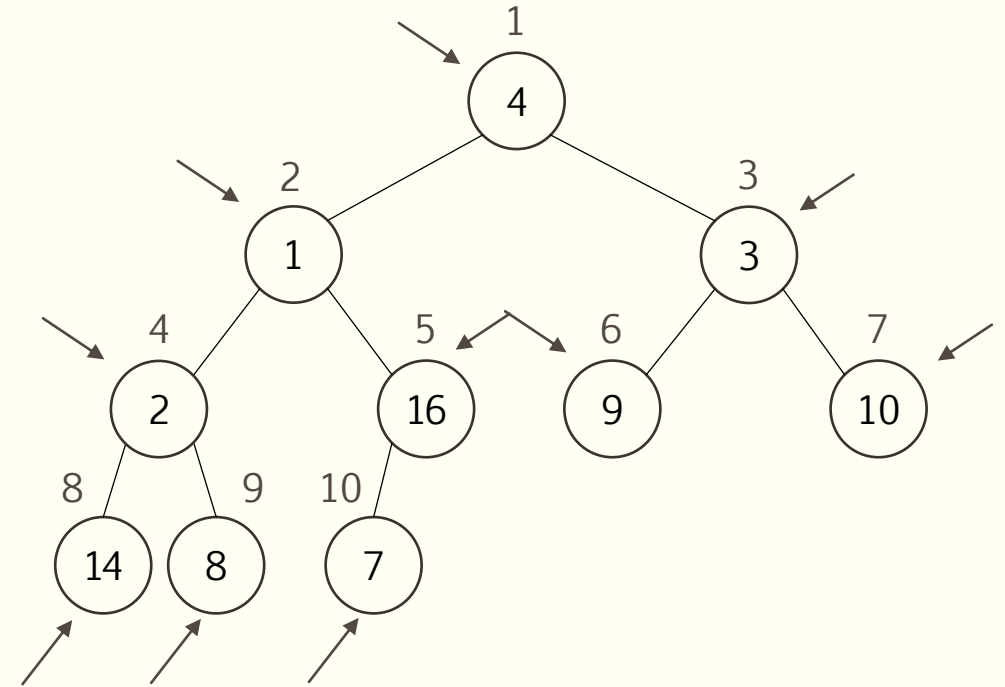
- Pour convertir un tableau $t[1..n]$ (avec $n = t.longueur$) en tas max
 - On utilise la propriété suivante
 - Les éléments du sous-tableau $t[(n / 2 + 1) \dots n]$ sont tous des feuilles de l'arbre, donc des tas max à 1 élément
 - On utilise la procédure Entasser-Max sur les autres nœuds de l'arbre, à l'envers

Exemple : Construire-Tas-Max avec $t = \langle 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 \rangle$

- À partir du tableau initial suivant

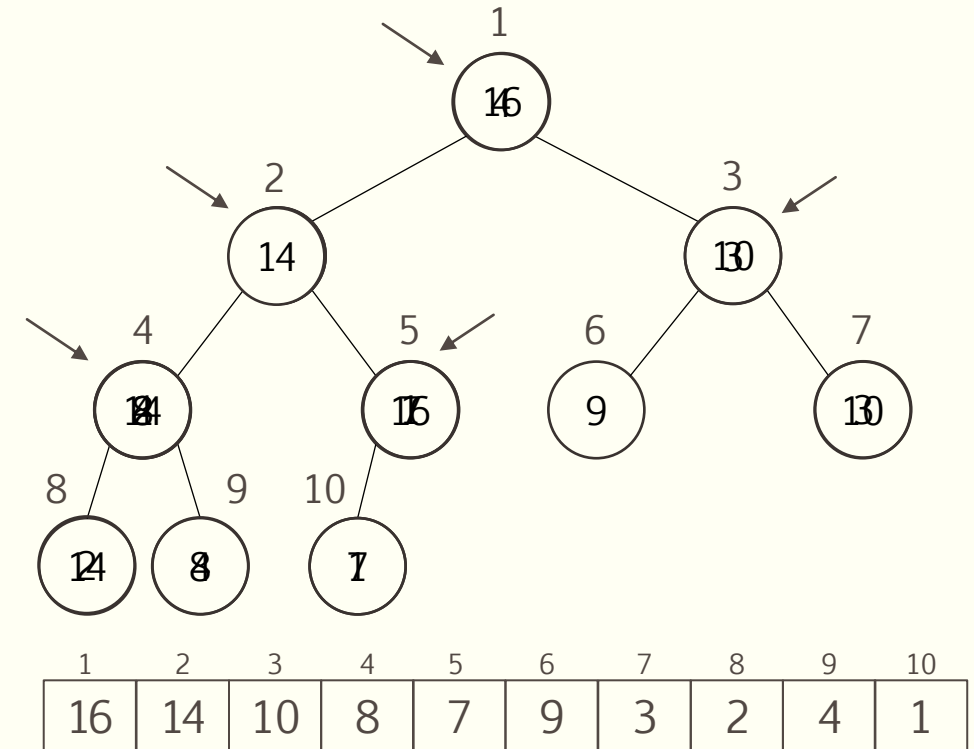
1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7
↑	↑	↑	↑	↑	↑	↑	↑	↑	↑

- On veut construire un tas max
- Les éléments situés dans la partie droite du tableau
 - À partir de l'indice $t.\text{longueur} / 2 + 1$
 - Sont des tas max à un élément
- On peut donc appeler Entasser-max sur le nœud d'indice $t.\text{longueur} / 2$
 - Et ensuite successivement sur les nœuds précédents jusqu'à la racine



Exemple : Construire-Tas-Max avec $t = \langle 4, 1, 3, 2, 16, 9, 10, 14, 8, 7 \rangle$

- On commence à l'indice 5
 - Entasser-Max : aucun changement
 - On appelle ensuite Entasser-Max à l'indice 4
 - On appelle ensuite Entasser-Max à l'indice 3
 - On appelle ensuite Entasser-Max à l'indice 2
 - On appelle ensuite Entasser-Max à l'indice 1
- L'algorithme est terminé : le tableau est maintenant structuré sous forme de tas max
- Fonction à définir : Construire-tas-Max



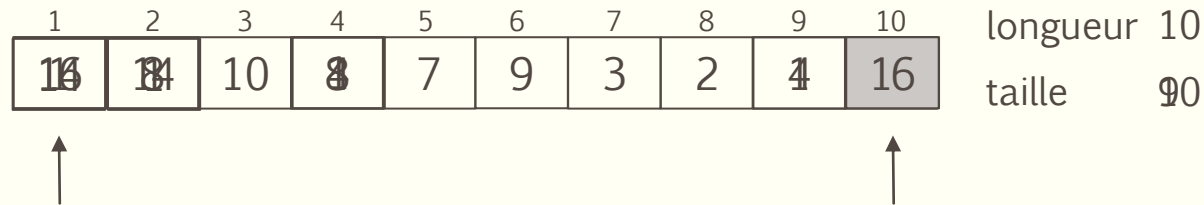
- Temps d'exécution : $O(n)$ appels à Entasser-Max
 - $O(n \lg n)$ (une analyse plus fine peut démontrer que c'est en fait $O(n)$)

Tri par tas

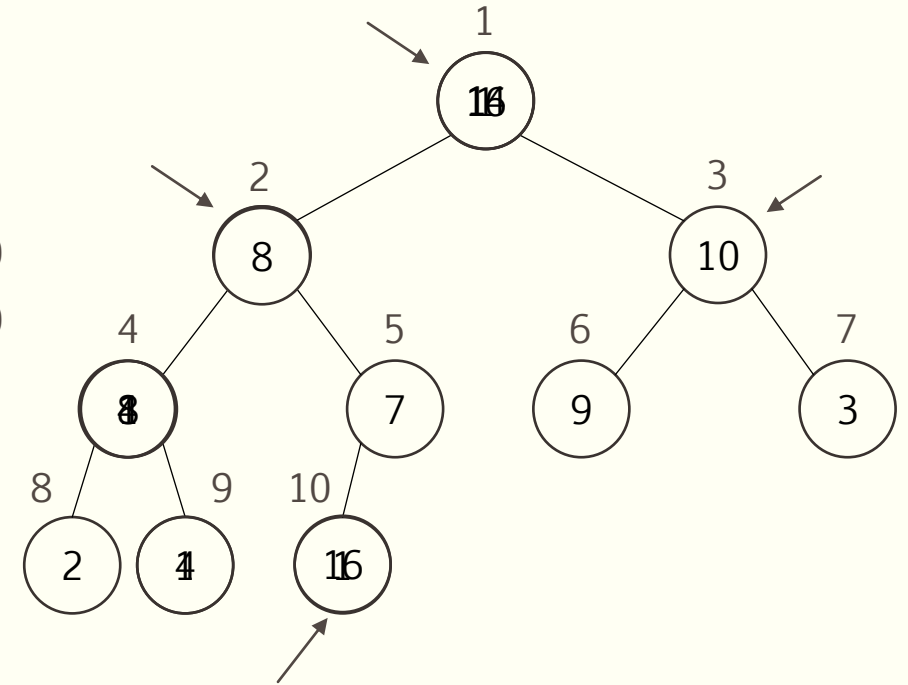
- Démarre en construisant un tas max avec le tableau d'entrée
- Principe
 - Élément maximal du tableau $\rightarrow$ racine de l'arbre $t[1]$
 - On peut donc le placer à sa position finale correcte, à la fin du tableau, en l'échangeant avec $t[n]$
 - ... et en enlevant le nœud n du tas en décrémentant $t.taille$
 - On rétablit ensuite la propriété de tas max en appelant Entasser-Max sur la racine
 - Et on répète le processus jusqu'à arriver à un tas de taille 1

Exemple : Tri-Par-Tas avec $t = \langle 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 \rangle$

- On veut trier un tableau à l'aide du tri par tas
- On construit un tas avec le tableau

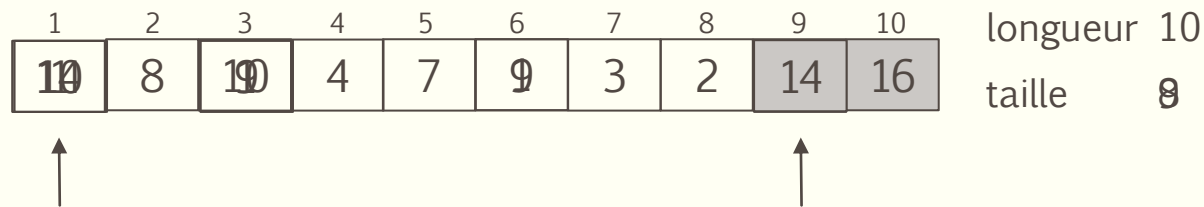


- La valeur maximale du tas se trouve à $t[1]$
 - Sa place finale devrait être à $t[10]$
 - Donc on échange $t[1]$ et $t[10]$
 - Et on retire $t[10]$ du tas
- La valeur à $t[1]$ viole la propriété de tas
 - $t[2]$ et $t[3]$ sont des tas
 - Donc on peut entasser $t[1]$

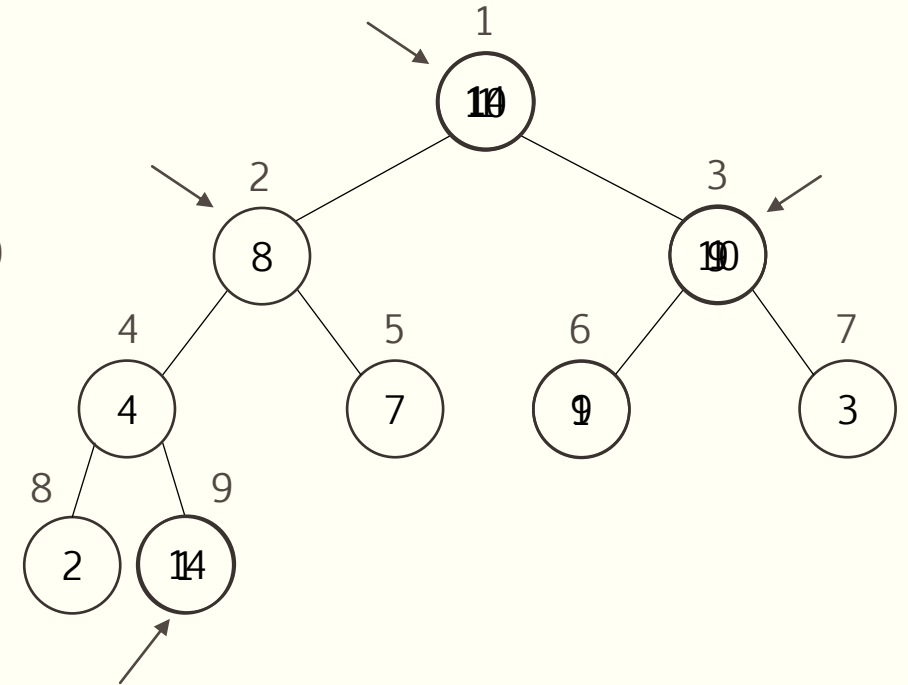


Exemple : Tri-Par-Tas avec $t = \langle 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 \rangle$

- On veut trier un tableau à l'aide du tri par tas
- On construit un tas avec le tableau



- La valeur maximale du tas se trouve à $t[1]$
 - Sa place finale devrait être à $t[9]$
 - Donc on échange $t[1]$ et $t[9]$
 - Et on retire $t[9]$ du tas
- La valeur à $t[1]$ viole la propriété de tas
 - $t[2]$ et $t[3]$ sont des tas
 - Donc on peut entasser $t[1]$



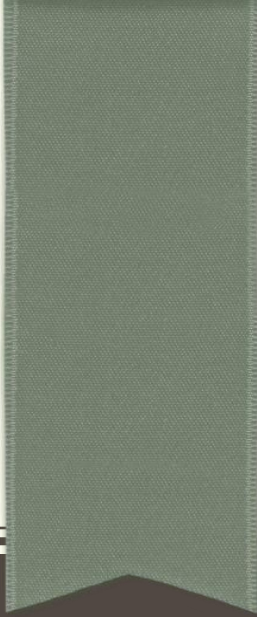
Exemple : Tri-Par-Tas avec $t = \langle 16, 14, 10, 8, 7, 9, 3, 2, 4, 1 \rangle$

- On veut trier un tableau à l'aide du tri par tas
- On construit un tas avec le tableau

1	2	3	4	5	6	7	8	9	10	longueur 10
1	2	3	4	7	8	9	10	14	16	taille 1

- À la fin de l'algorithme, le tableau est trié

- Construire-Tas-Max
 - $O(n \lg n)$ (ou $O(n)$, mais ça ne change pas le résultat)
- $O(n)$ appels à Entasser-Max
 - $O(n \lg n)$
- Temps d'exécution
 - $O(n \lg n)$
 - (Comme le tri par fusion)
- Trie sur place
 - (Comme le tri par insertion)



FIN DU FLASHBACK

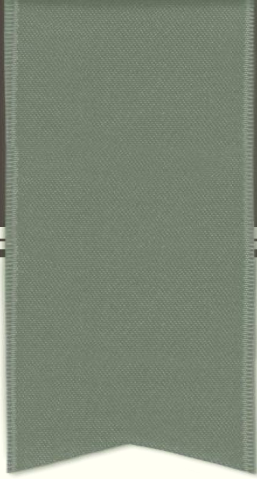
Tas

File de priorités

- Structure de données permettant de gérer un ensemble de tâches dont chacune possède une priorité
- Une file de priorités supporte les opérations suivantes :
 - Maximum : retourne la tâche ayant la priorité maximale
 - Extraire-max : supprime et retourne l'élément ayant la clé maximale
 - Augmenter-clé : accroît la valeur d'une tâche
 - Insérer : ajoute une nouvelle tâche
- On peut implémenter une file de priorités efficace en utilisant un tas
 - On verra comment en TD...

Algorithme de Prim : Mise en œuvre en utilisant une file de priorités

- On peut utiliser une file de priorités à la place du tableau d pour stocker les distances des sommets non encore couverts
 - La distance deviendra alors la priorité
 - On s'intéresse alors à la tâche de priorité minimale plutôt que maximale
- La file de priorités devra donc supporter les opérations suivantes :
 - Minimum : retourne l'élément ayant la priorité minimale
 - Extraire-min : supprime et retourne l'élément ayant la clé minimale
 - Diminuer-clé : diminue la valeur d'un élément
 - Insérer : ajoute un nouvel élément
- Il faudra alors utiliser un tas min plutôt qu'un tas max
- L'extraction du sommet de plus petite distance deviendra alors plus efficace
- Cette version de l'algorithme sera vue en TD



PROCHAIN COURS

STRUCTURES DE DONNÉES DYNAMIQUES